

Java for networks
TP 1: HTTP client/server with Java sockets

Instructor Linda Senigagliesi
Email linda.senigagliesi@ensea.fr

Students BECHAINE Douae
BENSLIMA Rania
Class RTS

Year 2026-2027

1 Client-server communication with netcat :

Step 1 :
Server (Linux): `netcat -l 10.10.26.248 5000`. Client (Windows): `ncat 10.10.26.248 5000`.

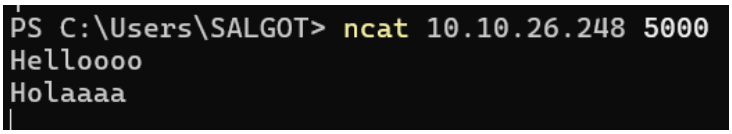


Figure 1: Client side: `ncat` connected to the server.

The option `-l` makes netcat bind to the port and listen for incoming connections.

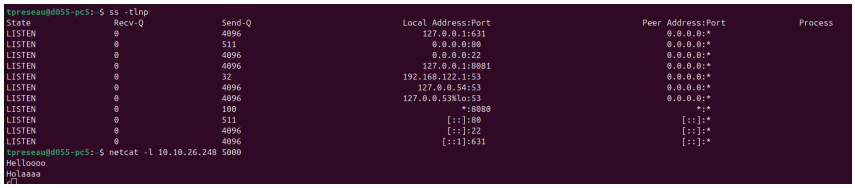


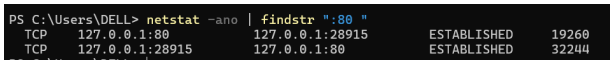
Figure 2: Server side: `ss -tlnp` and messages received.

Conclusion: both sides can send at any moment, so TCP is full-duplex: data flows in both directions independently and simultaneously.
Step 2:

I started ncat -l 80 and opened http://localhost in Chrome. The page displayed nothing, because the server in my terminal have not answered yet.the ncat terminal receive the HTTP request sent by the browser: the request line GET / HTTP/1.1, followed by headers (Host: localhost, Connection: keep-alive...) With netstat -ano, we saw an ESTABLISHED TCP connection between port 80 (ncat) and a random port chosen by the browser.



With netstat -ano, I saw an ESTABLISHED TCP connection between port 80 (ncat) and a random port chosen by the browser.





The client sends a request (method, resource, headers), and the server answers with a status line, headers and a body, separated by an empty line.ncat transports the bytes.

2 HTTP client :

Step 3:
We created the HttpClient class in the package fr.ensea.rts.http. This class will contain the methods needed to send an HTTP request and download a web page.

```
package fr.ensea.rts.http;

public class HttpClient {
```

Step 4:
We added five private attributes: 'protocol', 'host', 'port', 'path', and 'filename'. These variables will store the connection information and the name of the output file.

```
private String protocol;
private String host;
private int port;
private String path;
private String filename;
```

Step 5:
We created the 'readURL' method. It takes a URL as a String parameter. This method will be used to read and extract the information from the URL.

```
public void readURL(String in) {

}
```

Step 6 :
We used the URL class to extract the connection information :

```java
public static void readURL(String in) {
 try {
     URL url = URI.create(in).toURL();

     protocol = url.getProtocol();
     host = url.getHost();
     port = url.getPort();

     if (port == -1) {
         port = url.getDefaultPort();
     }

     path = url.getPath();

     if (path.isEmpty()) {
         path = "/";
     }

     filename = path.substring(path.lastIndexOf('/') + 1);

     if (filename.isEmpty()) {
         filename = "index.html";
     }

     System.out.println("Protocol: " + protocol);
     System.out.println("Host: " + host);
     System.out.println("Port: " + port);
     System.out.println("Path: " + path);
     System.out.println("Filename: " + filename);

 } catch (Exception e) {
     System.out.println("Error: " + e.getMessage());
 }
```

Figure 3: URL parsing code

The results show that the protocol is HTTP, the host is 'neverssl.com', and the default port is 80. Since the URL has no specific file path, the path is '/' and the output filename is 'index.html'. This confirms that the URL was parsed correctly.

```
PS C:\Users\SALGOT\HTTP-Lab> javac HttpClient.java
PS C:\Users\SALGOT\HTTP-Lab> java HttpClient http://neverssl.com/
Protocol: http
Host: neverssl.com
Port: 80
Path: /
Filename: index.html
```

Figure 4: URL parsing results

The results show that the protocol is HTTP, the host is 'neverssl.com', and the default port is 80. Since the URL has no specific file path, the path is '/' and the output filename is 'index.html'. This confirms that the URL was parsed correctly.

Step 7 :

We used Test-NetConnection to check the connection to neverssl.com on port 80. The test was successful (TcpTestSucceeded: True), while our Java program returned a timeout error. This shows that the server is reachable, but the Java connection failed. PowerShell used an IPv6 address, which may explain the difference.

```
PS C:\Users\SALGOT\HTTP-Lab> java HttpClient http://neverssl.com/
Protocol: http
Host: neverssl.com
Port: 80
Path: /
Filename: index.html
Connection error: Connection timed out: connect
```

Figure 5: TCP connectivity test

Step 8 :
We added Thread.sleep(30000) to keep the socket open for 30 seconds. We first ran our Java program with example.com on port 80, and the message Connected to example.com:80 confirmed that the connection was successful.

```
PS C:\Users\SALGOT\HTTP-Lab> java HttpClient http://example.com/
Protocol: http
Host: example.com
Port: 80
Path: /
Filename: index.html
Connected to example.com:80
```

Figure 6: Successful TCP connection to example.com

While the connection was open, we used netstat -ano to check its status. The result showed ESTABLISHED, meaning that the TCP connection was active. We then used Get-Process -Id 24900 to check the process ID. The result showed java, confirming that the connection belonged to our Java program.

```
PS C:\Users\SALGOT> netstat -ano | findstr ":80"
  TCP    192.168.32.182:1193    104.20.23.154:80       ESTABLISHED     24900
PS C:\Users\SALGOT> Get-Process -Id 24900

Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
    316      21   315352      42580       0,17  24900   3 java
```

Figure 7: TCP connection verification

These results confirm that our Java client successfully established and maintained a TCP connection with the web server.

Step 9 :
We created an InputStream to receive data from the server and a PrintWriter to send data to it. We tested the program with example.com, and both streams were created successfully.

```
PS C:\Users\SALGOT\HTTP-Lab> javac HttpClient.java
PS C:\Users\SALGOT\HTTP-Lab> java HttpClient http://example.com/
Protocol: http
Host: example.com
Port: 80
Path: /
Filename: index.html
Connected to example.com:80
InputStream and PrintWriter created successfully
```

Figure 8: Successful creation of input and output streams

The client is now ready to send an HTTP request.

Step 10 :
We used PrintWriter to send an HTTP GET request to example.com. The request includes the path, the host, and the Connection: close header. We used flush() to send the request. The results show that the TCP connection was established and the HTTP GET request was sent successfully.

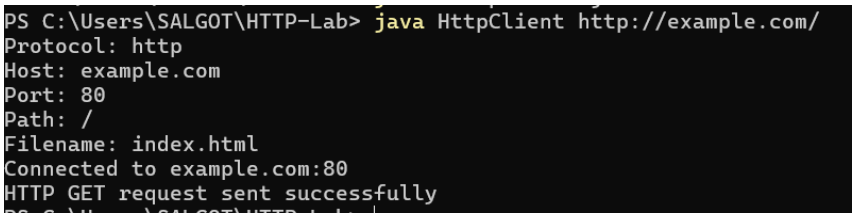


Figure 9: Sending an HTTP GET request

Step 11 :
We used 'FileOutputStream' to create a local file named 'index.html'. The PowerShell result confirms that the file was created successfully with a size of 0 bytes. This is expected because we have not saved the server's response yet.

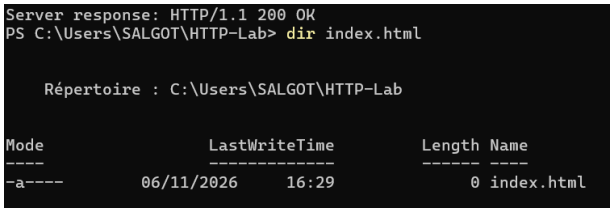


Figure 10: Verification of the created index.html file

Step 12 :
We used a while loop to read the server's response in blocks of 4096 bytes and save it to index.html. After opening the file, we found the HTTP status line 200 OK and the response headers. This confirms that the server's response was received and saved successfully.

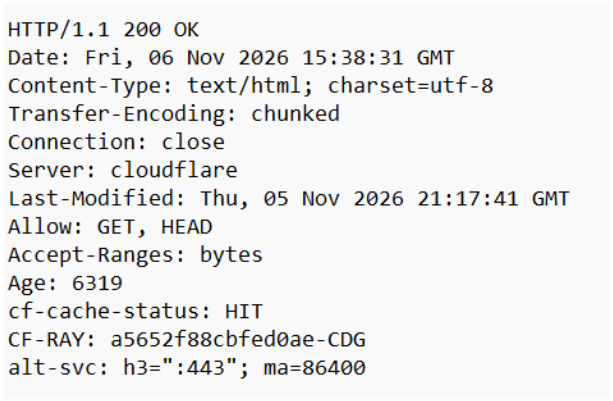


Figure 11: HTTP status line and response headers in index.html

We also checked the file size using the dir command. The size increased from 0 to 931 bytes, confirming that data was written to the file.

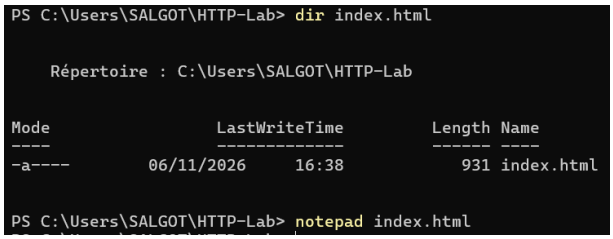


Figure 12: Verification of the saved file size (931 bytes)

Step 13 :
We used try-with-resources to automatically close the socket and the output file after saving the server's response. This ensures that the resources are properly released, even if an error occurs.

Step 14 :
We used BufferedReader and InputStreamReader to read the server's response, then readLine() to extract the HTTP status line :

```
BufferedReader reader = new BufferedReader(
    new InputStreamReader(from)
);

String statusLine = reader.readLine();

System.out.println("Server response: " + statusLine);
```

Figure 13: Java code for reading the HTTP response status line

The server returned 'HTTP/1.1 200 OK', which confirms that our GET request was successful. This shows that our client can send an HTTP request and receive a valid response.

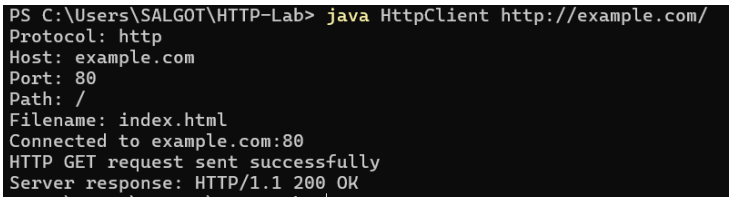


Figure 14: HTTP GET request and successful server response

However, the saved file still contains the HTTP headers, so we need to modify the code to save only the HTML content.
We modified the program to check the HTTP status code before saving the page. If the status is not 200, the program displays an error. We also skipped the HTTP headers and handled the chunked response to save only the HTML content. The results show HTTP/1.1 200 OK, and the program successfully saved 577 bytes to index.html.

```
Protocol: http
Host: example.com
Port: 80
Path: /
Filename: index.html
Connected to example.com:80
HTTP GET request sent successfully
Server response: HTTP/1.1 200 OK
File created: index.html
HTML saved to: index.html
Total bytes saved: 577
```

Figure 15: Successful HTTP Response and HTML File Download

When we opened the file, it started with <!doctype html> instead of the HTTP headers. This confirms that the program now saves only the web page content :

```
<!doctype html><html lang=en><head><meta charset=utf-8><link rel=icon href=data:,><meta name=viewport content="width=device-width,initial-
scale=1"><title>Example Domain</title><style>html{color-scheme:light dark;background:light-dark(#eee,#222)}body{font:16px/1.6 system-
ui,sans-serif;max-width:26em;margin:auto;padding:25vh 2em 2em;text-align:center}</style></head><body><p>This domain is for use in
documentation examples without needing permission. This is not a service; avoid relying on it for testing and monitoring purposes.</p>
<script src=/s.js></script></body></html>
```

Figure 16: HTML content saved without HTTP headers

Step 15 :

We used Wireshark to observe our Java client's network communication :

946	37.083721800	192.168.32.182	192.168.35.250	DNS	71 Standard query 0x0c68 A example.com
947	37.096573200	192.168.35.250	192.168.32.182	DNS	103 Standard query response 0x0c68 A example.com A 172.66.147.243 A 104.20.23.154
948	37.113862400	192.168.32.182	172.66.147.243	TCP	66 35734 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
961	38.117733300	192.168.32.182	172.66.147.243	TCP	66 [TCP Retransmission] 35734 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1025	40.126785900	192.168.32.182	172.66.147.243	TCP	66 [TCP Retransmission] 35734 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1028	40.134618400	172.66.147.243	192.168.32.182	TCP	66 80 → 35734 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 SACK_PERM WS=8192
1029	40.134985600	192.168.32.182	172.66.147.243	TCP	54 35734 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1030	40.142934200	192.168.32.182	172.66.147.243	HTTP	110 GET / HTTP/1.1
1031	40.151795900	172.66.147.243	192.168.32.182	TCP	60 80 → 35734 [ACK] Seq=1 Ack=57 Win=131072 Len=0
1032	40.157845100	172.66.147.243	192.168.32.182	TCP	980 80 → 35734 [PSH, ACK] Seq=1 Ack=57 Win=131072 Len=926 [TCP PDU reassembled in 1033]
1033	40.161549800	172.66.147.243	192.168.32.182	HTTP	60 HTTP/1.1 200 OK (text/html)
1034	40.161683700	192.168.32.182	172.66.147.243	TCP	54 35734 → 80 [ACK] Seq=57 Ack=933 Win=64512 Len=0
1035	40.167237700	192.168.32.182	172.66.147.243	TCP	54 35734 → 80 [FIN, ACK] Seq=57 Ack=933 Win=64512 Len=0
1036	40.167384900	192.168.32.182	172.66.147.243	TCP	54 35734 → 80 [RST, ACK] Seq=58 Ack=933 Win=0 Len=0

Figure 17: Wireshark TCP and HTTP Capture

In the capture, packets 946–947 show the DNS request and response for example.com. Packets 948, 1028 and 1029 show the TCP three-way handshake (SYN, SYN-ACK, ACK). We also observed SYN retransmissions in packets 961 and 1025 before the connection succeeded.

Once connected, packet 1030 shows the HTTP GET request, and packet 1033 shows the server's 200 OK response. Finally, packets 1035–1036 show a FIN-ACK followed by a RST-ACK from the client, marking the end of the connection.

This capture confirms that our Java client successfully established a TCP connection, requested the web page and received the server's response.

3 HTTP servers:

Step 17 :

On Windows, the whole configuration is in the file `C:\nginx\conf\nginx.conf` (there is no `sites-enabled` folder as on Debian/Ubuntu). The relevant part is the `server` block:

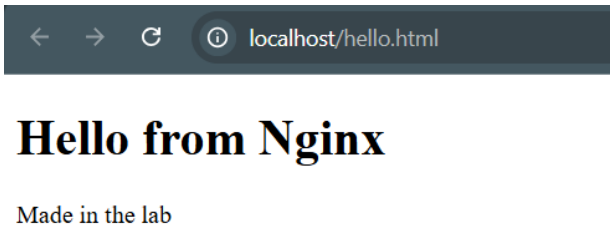
```
server {
    listen       80;
    server_name  localhost;

    #access_log  logs/host.access.log  main;

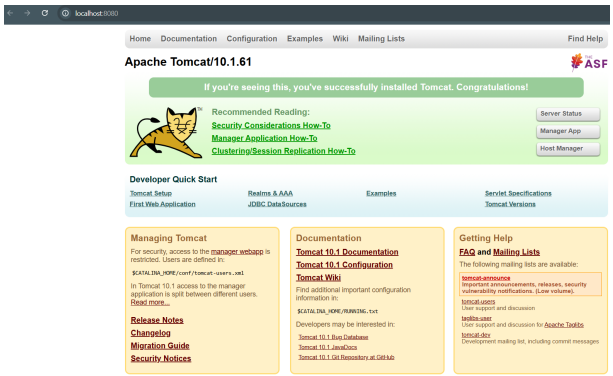
    location / {
        root   html;
        index  index.html index.htm;
    }
```

Step 18 :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Hello Nginx</title>
</head>
<body>
  <h1>Hello from Nginx</h1>
  <p>Made in the lab</p>
</body>
</html>
```



Step 19 :



Address: http://localhost:8080

Step 20 :



Upon clicking on JSP page



Step 21 :

We edited webapps/sample/hello.jsp in the Tomcat folder and added the following inside the <body>:

```
<% int a = 6, b = 7; %>
<p>6 x 7 = <%= a * b %></p>
<p>Server time: <%= new java.util.Date() %></p>
<% for (int i = 1; i <= 3; i++) { %>
  <p>Line number <%= i %></p>
<% } %>
```



- **Scriptlet** <% ... %>: Java code executed on the server.
- **Expression** <%= ... %>: a Java expression whose value is converted to text and inserted into the HTML.

unlike a static HTML file (like the one served by Nginx), a JSP is not sent as it is. whenever the file changes, Tomcat translates the JSP into a Java servlet and compiles it,thus the time change with the java.util.Date when we refech the page each time.

Step 22:

Step 22: Roles of Nginx and Tomcat

Nginx is a lightweight HTTP server. It serves **static files** (HTML, CSS, images) quickly, handles many simultaneous connections, and can act as a **reverse proxy** and load balancer. It cannot run Java code.

Tomcat is a **servlet container**. It runs Java web applications (servlets and JSP) and generates **dynamic content** at each request. It can serve files too, but less efficiently than Nginx.

In this lab: Nginx (port 80) returned `hello.html` as stored on disk, while Tomcat (port 8080) executed `hello.jsp` on the server.

Conclusion: the two are complementary. Nginx handles static content and traffic, and Tomcat runs the Java application logic.

Step 23: Linking Nginx and Tomcat

Nginx is configured as a **reverse proxy** in front of Tomcat. Clients only talk to Nginx (port 80). Nginx serves static files itself and forwards the other requests to Tomcat (port 8080).

After `nginx -s reload`, `http://localhost/sample/` is answered by Tomcat through Nginx, while `http://localhost/hello.html` is served directly by Nginx.

Advantages: a single entry point, Nginx handles static files, TLS and load balancing, and Tomcat stays hidden behind it.